

Lab 1 – Setting Up Flutter and Running Your First App

1. Objective

This lab helps students set up the Flutter environment and run their first Flutter application. Students will learn how to install tools, create a project, execute it on a device/emulator, and perform basic UI customization.

2. Requirements

Students must:

- Install Flutter SDK
- Configure IDE (Android Studio or VS Code)
- Create and run a Flutter app
- Customize the layout
- Submit screenshots as evidence

3. Guided Steps

Exercise 1 – Flutter Environment Setup

Goal: Install and verify Flutter tools.

Steps

1. Install Flutter SDK.
2. Add Flutter to PATH.
3. Run:

`flutter doctor`

4. Ensure all items are properly configured.
5. Verify Flutter & Dart plugins in Android Studio.

Deliverables

- Screenshot of flutter doctor
- Screenshot of Flutter/Dart plugin screen

Exercise 2 – Create and Run Your First Flutter App

Goal: Learn how to create a Flutter project and run the default app.

Steps

1. Open Android Studio → **New Flutter Project**
2. Use project name: `hello_flutter_lab1`
3. Explore key folders: `lib/`, `android/`, `ios/`, `pubspec.yaml`
4. Start emulator or connect device
5. Run the default counter app
6. Modify the AppBar title in `main.dart`:

title: 'My First Flutter App',

7. Use **Hot Reload** and observe changes

Deliverables

- Screenshot of project structure
- Screenshot of running counter app
- Screenshot showing updated title via Hot Reload

Exercise 3 – Customize Your First Flutter UI

Goal: Build a simple custom UI with widgets learned in class.

Steps

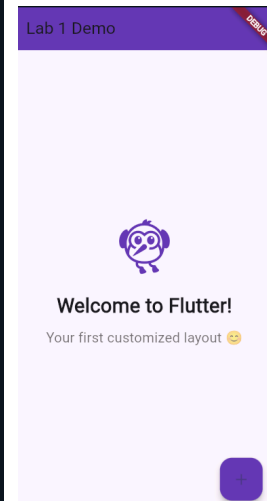
1. Replace default code in main.dart with:

```
import 'package:flutter/material.dart';

void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Lab 1 Demo'),
          backgroundColor: Colors.deepPurple,
        ),
        body: const Center(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              Icon(Icons.flutter_dash, size: 80, color: Colors.deepPurple),
              SizedBox(height: 20),
              Text(
                'Welcome to Flutter!',
                style: TextStyle(fontSize: 26, fontWeight: FontWeight.bold),
              ),
              SizedBox(height: 10),
              Text(
                'Your first customized layout 😊',
                style: TextStyle(fontSize: 18, color: Colors.black54),
              ),
            ],
          ),
        ),
        floatingActionButton: FloatingActionButton(
          onPressed: null,
          backgroundColor: Colors.deepPurple,
          child: Icon(Icons.add),
        ),
      ),
    );
  }
}
```



2. Perform Hot Reload and verify UI updates.
3. Modify colors/text and reload again.

Exercise 4 – Reflection Questions

1. What is the purpose of the flutter doctor command?
2. What file acts as the entry point of a Flutter application?
3. Explain the difference between **Hot Reload** and **Hot Restart**.

4. How does runApp() build the widget tree?
5. Describe how Flutter's architecture enables cross-platform development.

4. Expected Results

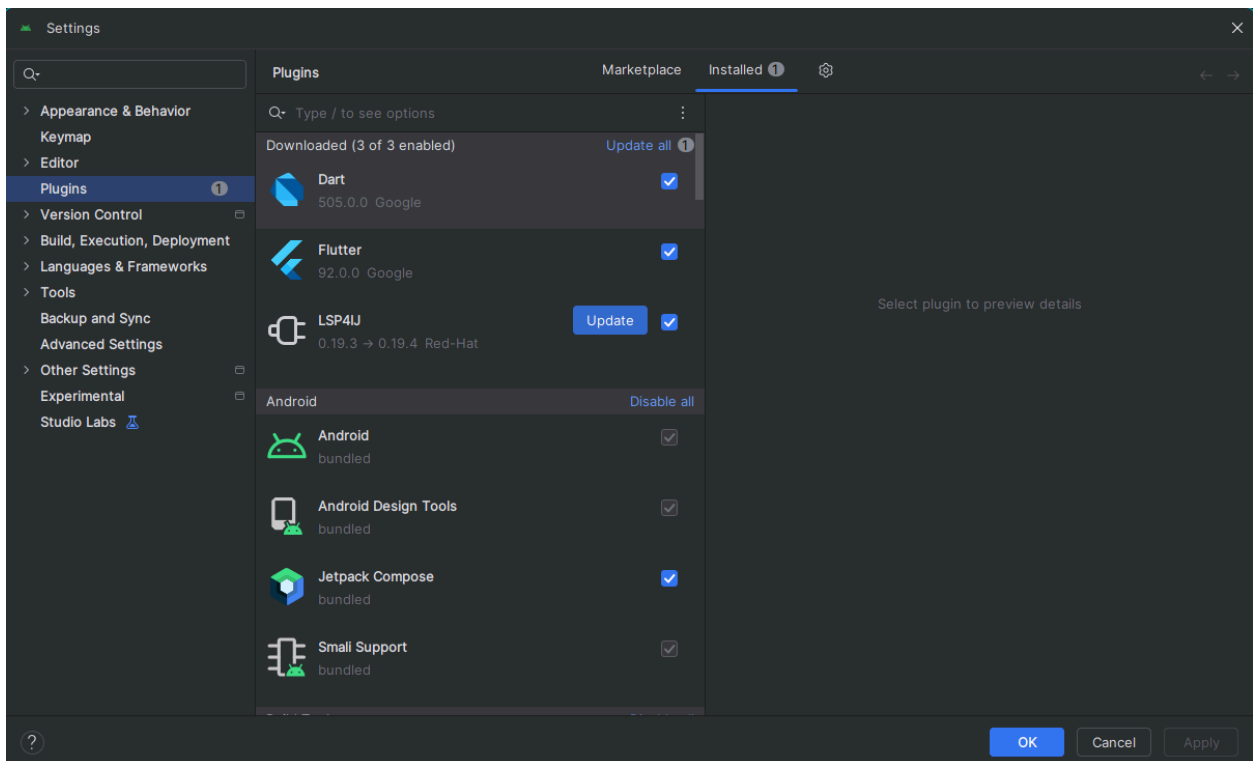
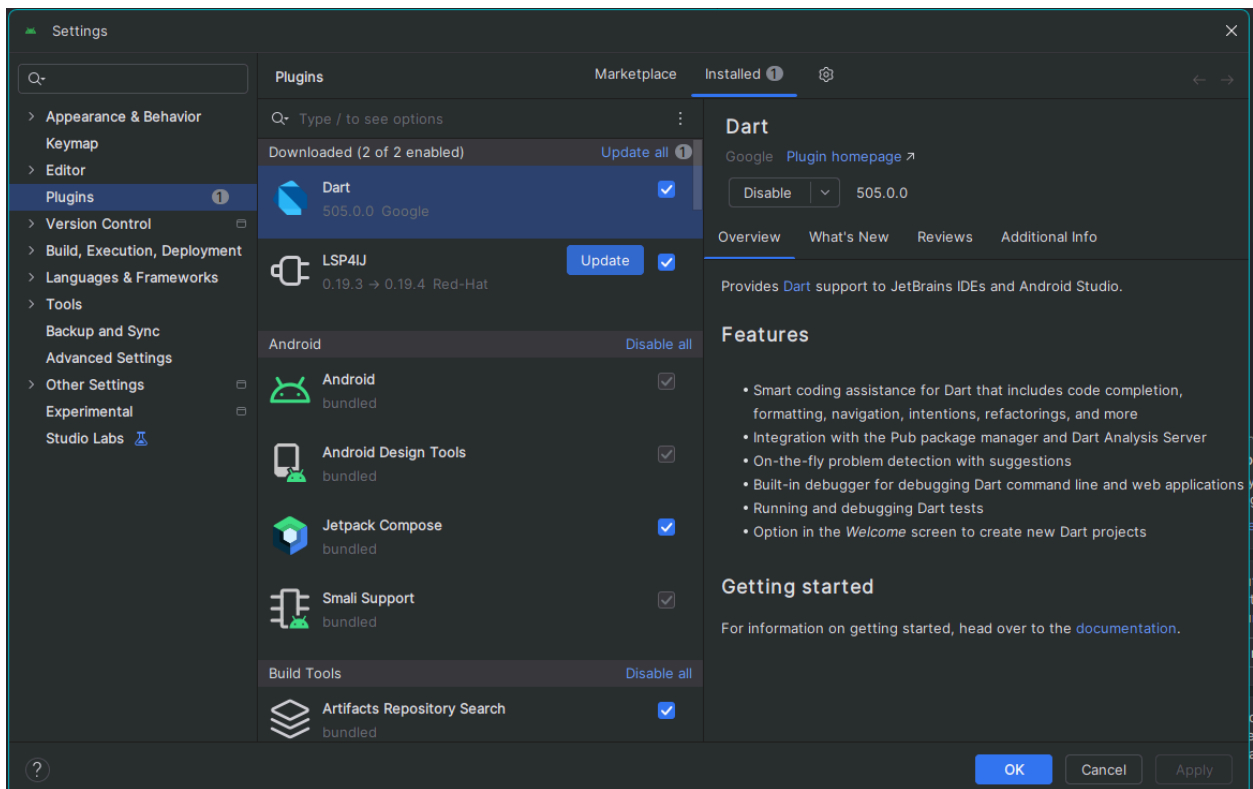
- Successful setup of Flutter environment
- Ability to create and run Flutter projects
- Understanding of widgets, layout, and hot reload
- Complete screenshots for all required steps

5. Submission

- Zipped Flutter project folder or GitHub link
- All screenshots
- Written answers for reflection questions

Nộp bài :

1.



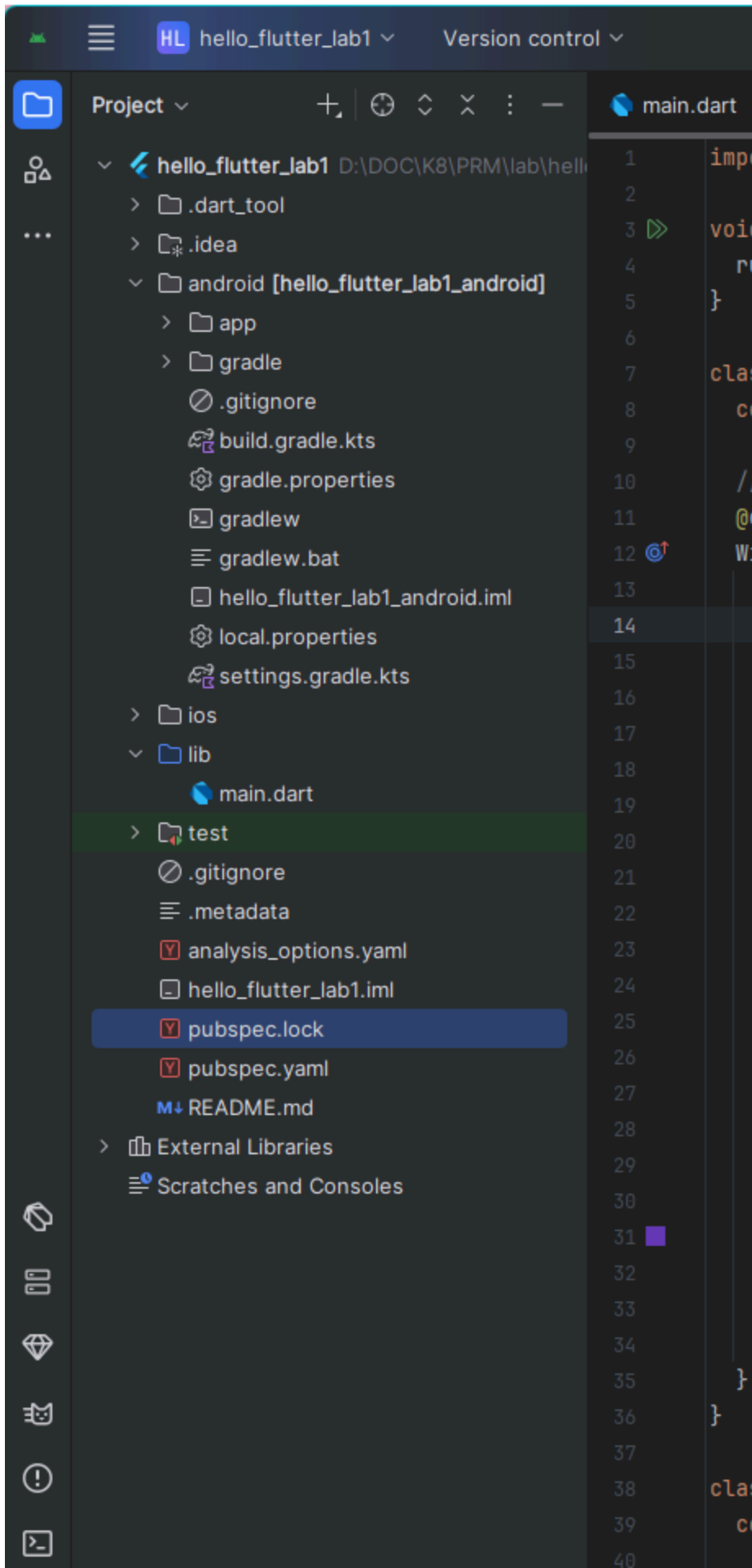
```
Command Prompt - flutter doctor
Microsoft Windows [Version 10.0.26200.8457]
(c) Microsoft Corporation. All rights reserved.

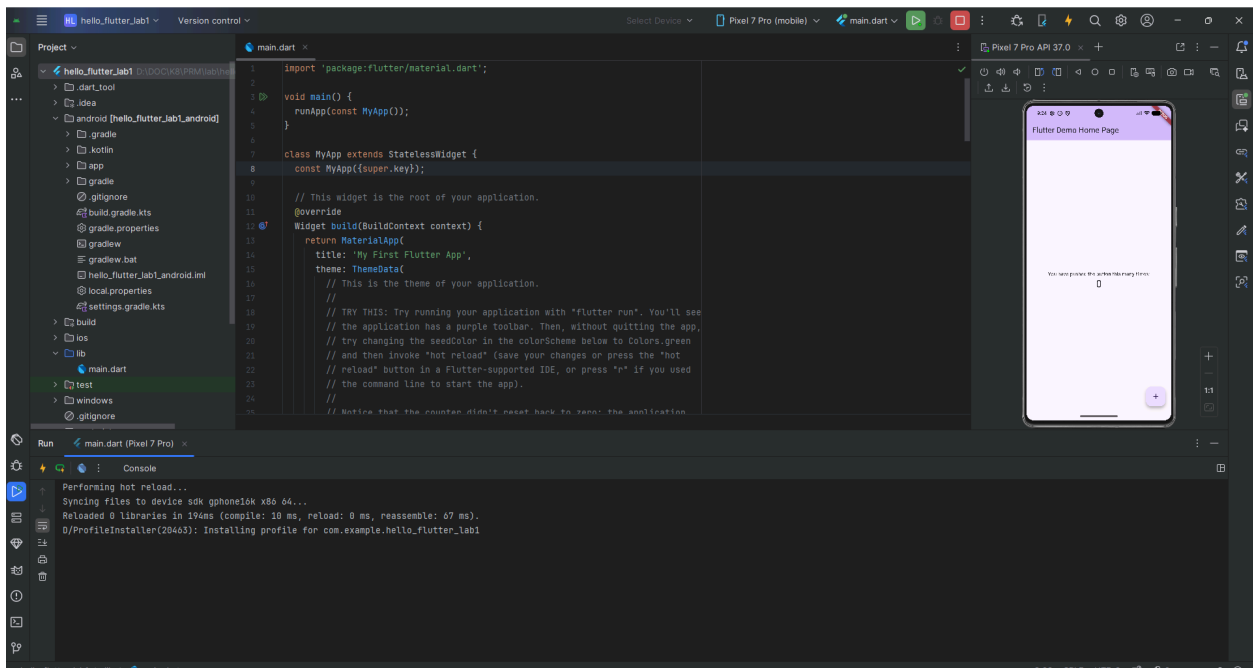
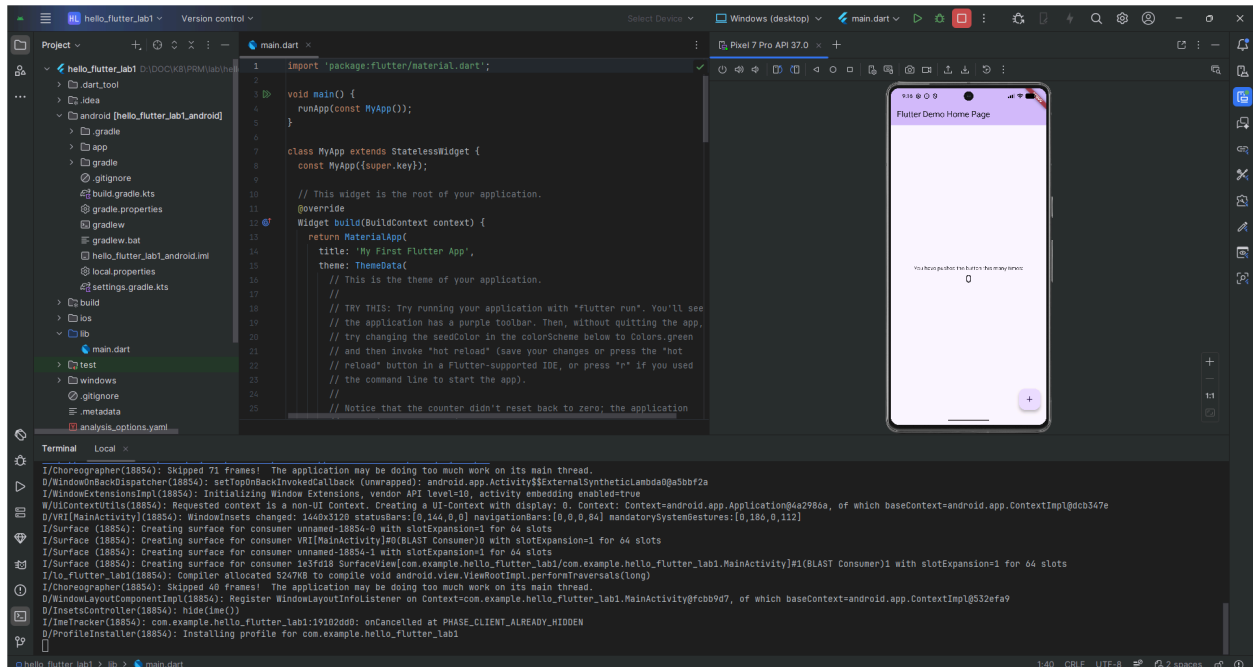
C:\Users\truon>flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[V] Flutter (Channel stable, 3.38.7, on Microsoft Windows [Version 10.0.26200.8457], locale en-US)
[V] Windows Version (Windows 11 or higher, 25H2, 2009)
[V] Android toolchain - develop for Android devices (Android SDK version 36.1.0)
[V] Chrome - develop for the web
[!] Visual Studio - develop Windows apps (Visual Studio Community 2022 17.14.16)
    X Visual Studio is missing necessary components. Please re-run the Visual Studio installer for the "Desktop
      development with C++" workload, and include these components:
        MSVC v142 - VS 2019 C++ x64/x86 build tools
        - If there are multiple build tool versions available, install the latest
        C++ CMake tools for Windows
        Windows 10 SDK
[V] Connected device (3 available)
[V] Network resources

! Doctor found issues in 1 category.

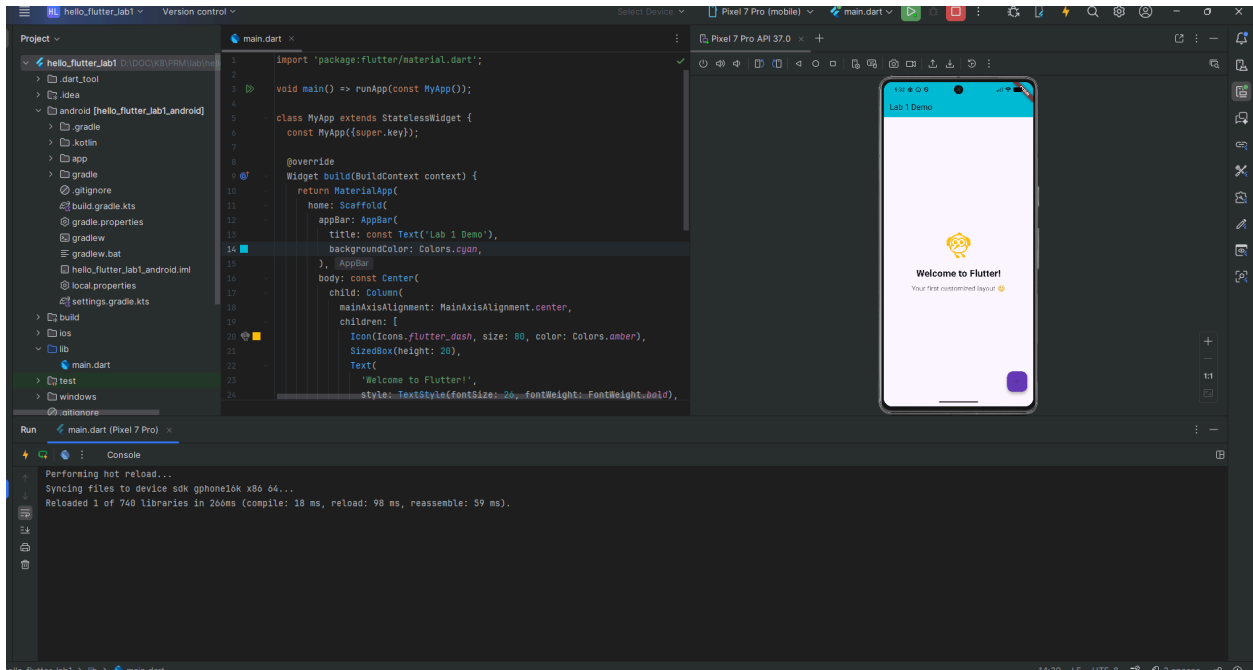
C:\Users\truon>
```

2.





3.



4.

Exercise 4:

1. What is the purpose of the flutter doctor command?

It diagnoses your local development environment. It checks if the required tools, SDKs, IDEs, and dependencies (like the Flutter SDK, Android Studio, Xcode, VS Code, and connected devices) are correctly installed and configured, and reports any missing setups.

2. What file acts as the entry point of a Flutter application?

The entry point file is `lib/main.dart` (specifically the `main()` function inside it).

3. Explain the difference between **Hot Reload** and **Hot Restart**.

- **Hot Reload:** Injects updated source code files into the running Dart Virtual Machine (VM). It updates the UI quickly (usually under a second) while preserving the app state (e.g., text inputs, counter values).
- **Hot Restart:** Rebuilds the widget tree from scratch by reloading the code and resetting the app state to its initial conditions. It takes slightly longer than Hot Reload.

4. How does `runApp()` build the widget tree?

`runApp()` takes a root widget as its argument, attaches it to the screen, and boots up the Flutter framework. It creates the root of the widget tree (specifically initializing the `RenderObjectToWidgetAdapter`), which in turn creates the corresponding `Element` and `RenderObject` trees to render the user interface.

5. Describe how Flutter's architecture enables cross-platform development.

Flutter does not rely on platform-native UI components. Instead, it compiles to native machine code (ARM or x86) and uses its own high-performance rendering engine (Impeller/Skia) to draw every pixel of the UI directly onto a platform canvas. This architectural approach ensures that the same code renders identically and runs with high performance across Android, iOS, web, and desktop platforms.